

Conception d'un analyseur lexical simplifié à l'aide d'automates à états finis

Objectifs pédagogiques

À la fin de ce TP, l'étudiant devra être capable de :

- Comprendre le principe de l'analyse lexicale.
- Identifier les différentes catégories de lexèmes (tokens) dans un programme.
- Construire un automate à états finis pour reconnaître des mots appartenant à un langage donné.
- Implémenter un automate en utilisant un langage de programmation (Python, C ou Java).
- Simuler le fonctionnement d'un analyseur lexical sur un texte source.

Mise en situation

On souhaite développer un petit analyseur lexical pour un langage de programmation très simplifié. L'analyseur doit lire une chaîne de caractères et identifier les éléments suivants :

Catégorie	Exemples
Identificateurs	x, somme, var1
Nombres entiers	12, 345, 0
Opérateurs arithmétiques	+, -, *, /
Opérateur d'affectation	=
Séparateurs	(,), ;

On adoptera les conventions suivantes :

- Un identificateur commence par une lettre et peut être suivi de lettres ou de chiffres.
- Un nombre entier est constitué d'une ou plusieurs chiffres.
- Les espaces et les tabulations sont ignorés.
- Les caractères non reconnus doivent être signalés comme des erreurs lexicales.

Partie 1 - Compréhension de l'analyse lexicale

Considérons le programme suivant :

```
somme = valeur1 + 25;\nresultat = somme * 2;
```

- ☐ Qu'est-ce qu'un lexème ?
- ☐ Identifier tous les lexèmes présents dans ce programme.
- ☐ Associer chaque lexème à sa catégorie lexicale (identificateur, nombre, opérateur, séparateur).

Réponses :

.....

Partie 2 - Construction des automates

Automate A : reconnaissance des identificateurs

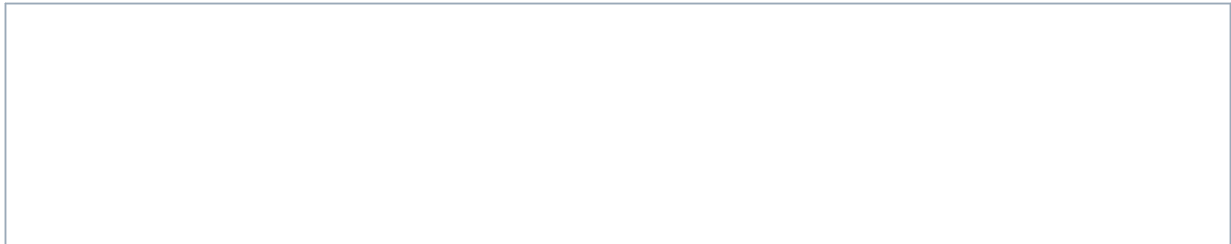
Rappel : un identificateur commence par une lettre, suivie éventuellement de lettres ou de chiffres.

Exemples acceptés : x, nom, var1, somme25.

Exemples refusés : 25var, 1abc, _nom.

- ☐ Définir les états de l'automate.
- ☐ Identifier l'état initial et les états finaux.
- ☐ Dessiner le graphe de transition.
- ☐ Construire la table de transition.

Espace pour dessiner l'automate A :



Partie 3 - Implémentation des automates

En utilisant Python (ou un autre langage), implémenter les deux automates précédents. Chaque fonction doit recevoir une chaîne de caractères et retourner True si elle est reconnue, et False dans le cas contraire.

Exemple de fonction attendue :

```
def reconnaitre_identificateur(mot):
    etat = 0

    for caractere in mot:
        if etat == 0:
            if caractere.isalpha():
                etat = 1
            else:
                return False

        elif etat == 1:
            if caractere.isalnum():
                etat = 1
            else:
                return False

    return etat == 1
```

Travail demandé :

- ☐ Écrire une fonction similaire pour reconnaître les nombres entiers.
- ☐ Tester les fonctions avec les exemples des parties précédentes.
- ☐ Proposer au moins trois tests supplémentaires pour chaque automate.

Tableau de tests :

Chaîne testée	Automate testé	Résultat obtenu

Partie 4 - Réalisation d'un analyseur lexical simplifié

Il s'agit maintenant de combiner les automates précédents pour construire un petit analyseur lexical. L'analyseur devra :

- Lire une chaîne de caractères représentant un programme.
- Parcourir cette chaîne caractère par caractère.
- Identifier les lexèmes.
- Reconnaître leur catégorie à l'aide des automates.
- Afficher la liste des tokens reconnus.

Exemple d'entrée :

x = 25 + somme2;

Résultat attendu :

Lexème	Token
x	IDENTIFICATEUR
=	AFFECTATION

25	NOMBRE
+	OPERATEUR
somme2	IDENTIFICATEUR
;	SEPARATEUR

L'analyseur devra également signaler les erreurs lexicales.

Exemple :

x = 25 + @;

Résultat attendu :

IDENTIFICATEUR : x
AFFECTATION : =
NOMBRE : 25
OPERATEUR : +
ERREUR LEXICALE : @

Travail demandé :

- ☐ Écrire l'algorithme général de parcours du texte source.
- ☐ Implémenter l'analyseur lexical et afficher les lexèmes avec leurs tokens.
- ☐ Tester l'analyseur sur les deux exemples et sur une entrée de votre choix.
- ☐ Expliquer comment l'analyseur réagit face à un caractère inconnu.

Espace pour l'algorithme et les remarques :